# Bloc 2 - Développement d'applications, API REST

## TD4– Sécuriser l'API avec Security et JWT

**Prérequis** : TD publier une API REST avec Symfony,
cours sécuriser l'authentification d'un projet Symfony avec Security

## Table des matières


1. Objectif ..... 1
2. Rappels sur Symfony, API Platform et le bundle Sécurity ..... 1
3. Travail à faire ..... 3
3.1 Préparation du projet ..... 3
3.2 Installation du bundle JWT Authentification ..... 8
3.3 Génération du token d'authentification ..... 11
3.4 Tester une consommation de l'API en passant le token ..... 15
3.5 Gestion des droits d'accès ..... 19
4. Conclusion ..... 26


## 1. Objectif

L'objectif de cette activité est de mettre en place un mécanisme d'authentification sécurisé sur une API REST développée avec le framework Symfony, son ORM doctrine et exposée via API Platform.

L'activité consiste à restreindre l'accès à une ressource accessible publiquement en implémentant une authentification basée sur le protocole JWT (JSON Web Token) à l'aide du composant Security de Symfony et du bundle LexikJWTAuthenticationBundle.

À l'issue de cette activité, l'API ne sera plus accessible anonymement, seules les requêtes authentifiées via un token valide pourront accéder aux ressources protégées.

## 2. Rappels sur Symfony, API Platform et le bundle Sécurity

***Symfony***
Symfony est un ensemble de composants PHP, un framework d'application Web, une philosophie et une communauté - tous travaillant ensemble en harmonie.
https://symfony.com/

***API Platform***
API Platform est un framework basé sur Symfony (et donc MVC) qui permet de développer simplement et rapidement une API REST.
https://api-platform.com/docs/core/getting-started/

***Le mécanisme d'authentification***
Le mécanisme d'authentification consiste à établir un lien irréfutable entre une personne et une identité numérique.

***Le Security Bundle***

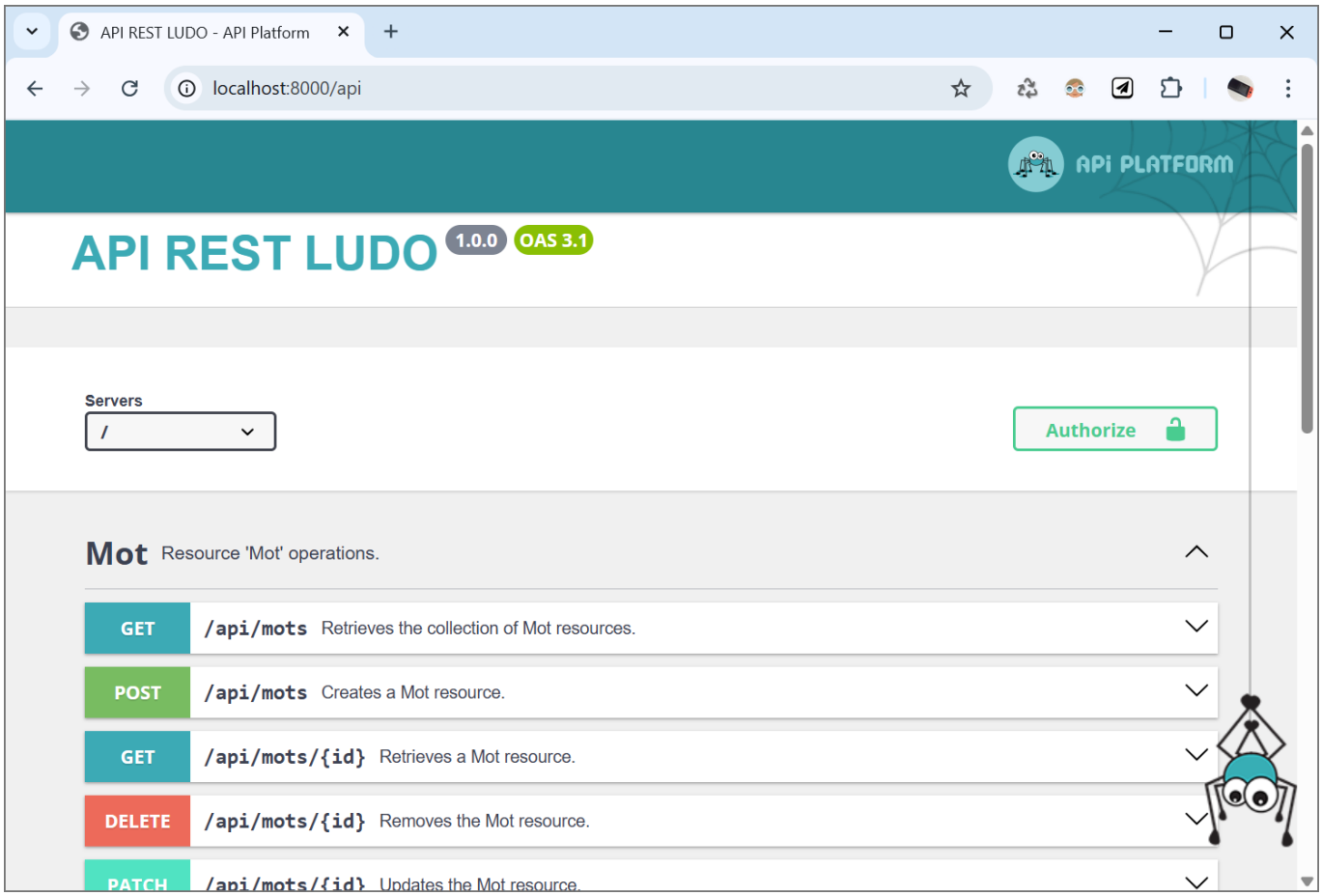
Le Security Bundle est le composant officiel de Symfony dédié à la gestion de la sécurité et de l'authentification des applications web.

Il fournit un ensemble complet de fonctionnalités permettant de contrôler l'accès aux ressources, d'authentifier les utilisateurs et de gérer leurs rôles et permissions.
Le Security Bundle s'intègre naturellement au reste de l'écosystème Symfony et constitue la base pour mettre en place un système d'authentification robuste, extensible et conforme aux bonnes pratiques de sécurité web.
👁 *https://symfony.com/doc/current/security.html*

# 3. Travail à faire

## 3.1 Préparation du projet

Démarrer le serveur Symfony, vérifier le bon fonctionnement de API Platform et les points de terminaison exposés :




Le système d'authentification va se baser sur le composant Symfony Security et une entité « User » dédié à la connexion.

Installer le composant Security de Symfony :

```
composer require symfony/security-bundle
```

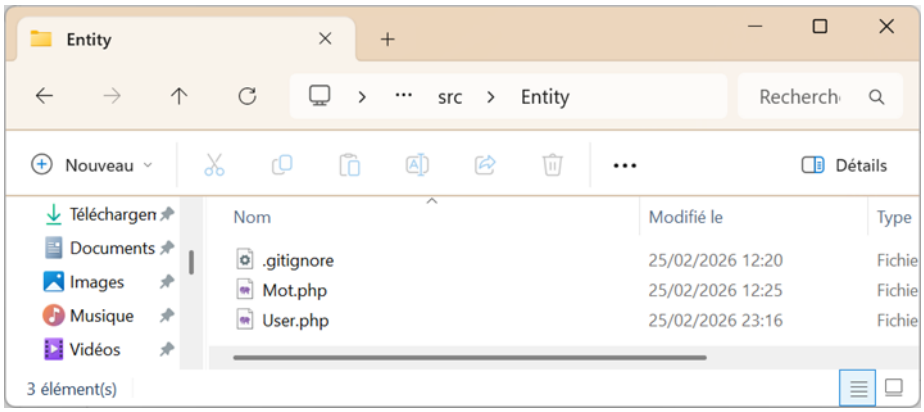
```
C:\wamp64\www\ludo-api>composer require symfony/security-bundle
./composer.json has been updated
Running composer update symfony/security-bundle
Loading composer repositories with package information
Updating dependencies
Nothing to modify in lock file
Writing lock file
Installing dependencies from lock file (including require-dev)
Nothing to install, update or remove
Generating autoload files
64 packages you are using are looking for funding.
Use the `composer fund` command to find out more!

Run composer recipes at any time to see the status of your Symfony recipes.

Executing script cache:clear [OK]
Executing script assets:install public [OK]

No security vulnerability advisories found.
```

Créer l'entité « User » nécessaire au futur système d'authentification en validant les choix proposés par défaut :

```
php bin/console make:user
```

```
C:\Windows\System32\cmd.e

C:\wamp64\www\ludo-api>php bin/console make:user

 The name of the security user class (e.g. User) [User]:
 >

 Do you want to store user data in the database (via Doctrine)? (yes/no) [yes]:
 >

 Enter a property name that will be the unique "display" name for the user (e.g. email,
 username, uuid) [email]:
 >

 Will this app need to hash/check user passwords? Choose No if passwords are not needed
 or will be checked/hashed by some other system (e.g. a single sign-on server).

 Does this app need to hash/check user passwords? (yes/no) [yes]:
 >

 created: src/Entity/User.php
 created: src/Repository/UserRepository.php
 updated: src/Entity/User.php
 updated: config/packages/security.yaml

 Success!

 Next Steps:
   - Review your new App\Entity\User class.
   - Use make:entity to add more fields to your User entity and then run make:migration
.
```

Vérifier la création du nouveau fichier src/Entity/User.php

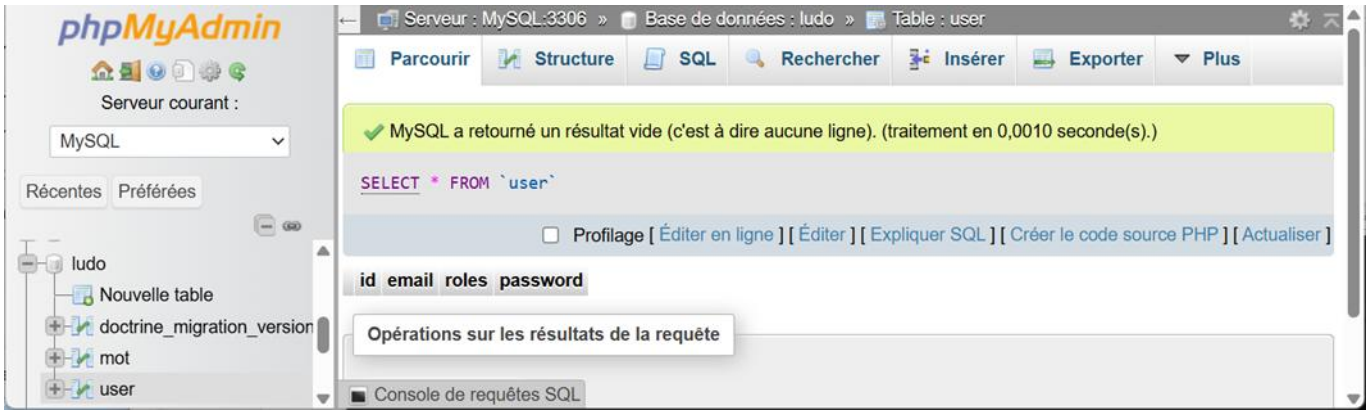



Migrer la création de l'entité User en BDD :

```
php bin/console make:migration
php bin/console doctrine:migrations:migrate
```

```
C:\wamp64\www\ludo-api>php bin/console make:migration
 created: migrations/Version20260225221848.php

  Success!

 Review the new migration then run it with php bin/console doctrine:migrations:migrate
 See https://symfony.com/doc/current/bundles/DoctrineMigrationsBundle/index.html

C:\wamp64\www\ludo-api>php bin/console doctrine:migrations:migrate

 WARNING! You are about to execute a migration in database "ludo" that could result in
schema changes and data loss. Are you sure you wish to continue? (yes/no) [yes]:
 >

[notice] Migrating up to DoctrineMigrations\Version20260225221848
[notice] finished in 299.1ms, used 20M memory, 1 migrations executed, 1 sql queries

 [OK] Successfully migrated to version: DoctrineMigrations\Version20260225221848
```

Vérifier la création de la table user correspondant à l'entity User en BDD




Créer un utilisateur en BDD via un DataFixtures, ce qui présente les avantages suivants :

- Il ne sera pas nécessaire de faire l'inscription,
- le compte sera créé immédiatement,
- il sera possible de réinitialiser le compte à tout moment.

Commencer par installer le composant ORM-fixtures :

```
composer require --dev orm-fixtures
```

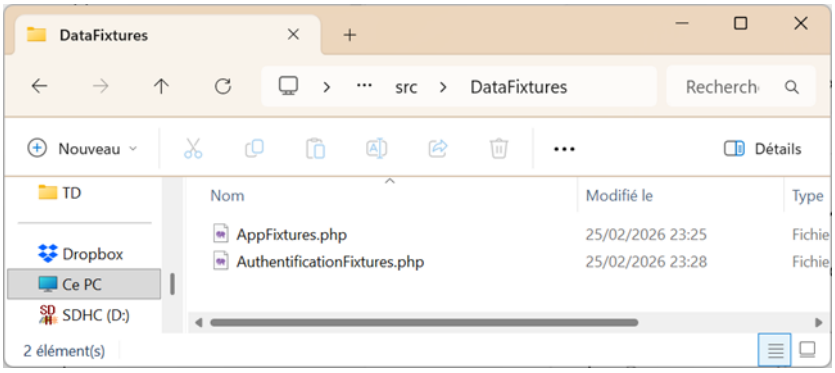
```
C:\wamp64\www\ludo-api>composer require --dev orm-fixtures
./composer.json has been updated
Running composer update doctrine/doctrine-fixtures-bundle
Loading composer repositories with package information
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
  - Locking doctrine/data-fixtures (2.2.0)
  - Locking doctrine/doctrine-fixtures-bundle (4.3.1)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 2 installs, 0 updates, 0 removals
  - Downloading doctrine/data-fixtures (2.2.0)
  - Downloading doctrine/doctrine-fixtures-bundle (4.3.1)
  - Installing doctrine/data-fixtures (2.2.0): Extracting archive
  - Installing doctrine/doctrine-fixtures-bundle (4.3.1): Extracting archive
Generating autoload files
66 packages you are using are looking for funding.
Use the `composer fund` command to find out more!

Symfony operations: 1 recipe (cea28b4f5b7acbe4e230a29faac2cb18)
  - Configuring doctrine/doctrine-fixtures-bundle (>=3.0): From github.com/symfony/reci
pes:main
Executing script cache:clear [OK]
Executing script assets:install public [OK]

 What's next?


Some files have been created and/or updated to configure your new packages.
Please review, edit and commit them: these files are yours.
```

Passer la commande suivante qui permet de créer un squelette de document fixtures à customiser :

```
php bin/console make:fixtures
```

```
 The class name of the fixtures to create (e.g. AppFixtures):
 > AuthentificationFixtures

 created: src/DataFixtures/AuthentificationFixtures.php

 Success!


 Next: Open your new fixtures class and start customizing it.
 Load your fixtures by running: php bin/console doctrine:fixtures:load
 Docs: https://symfony.com/doc/current/bundles/DoctrineFixturesBundle/index.html
```

Vérifier la création du fichier /src/DataFixtures/AuthentificationFixtures.php




Editer le fichier AuthentificationFixtures.php

```php
<?php
namespace App\DataFixtures;

use App\Entity\User;
use Doctrine\Bundle\FixturesBundle\Fixture;
use Doctrine\Persistence\ObjectManager;
use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;

class AuthentificationFixtures extends Fixture
{
    private UserPasswordHasherInterface $passwordHasher;

    public function __construct(UserPasswordHasherInterface $passwordHasher)
    {
        $this->passwordHasher = $passwordHasher;
    }

    public function load(ObjectManager $manager): void
    {
        $user = new User();

        $user->setEmail('prof@test.com');
        $user->setRoles(['ROLE_USER']);

        $hashedPassword = $this->passwordHasher->hashPassword(
            $user,
            '123456'
        );

        $user->setPassword($hashedPassword);

        $manager->persist($user);
        $manager->flush();
    }
}
```

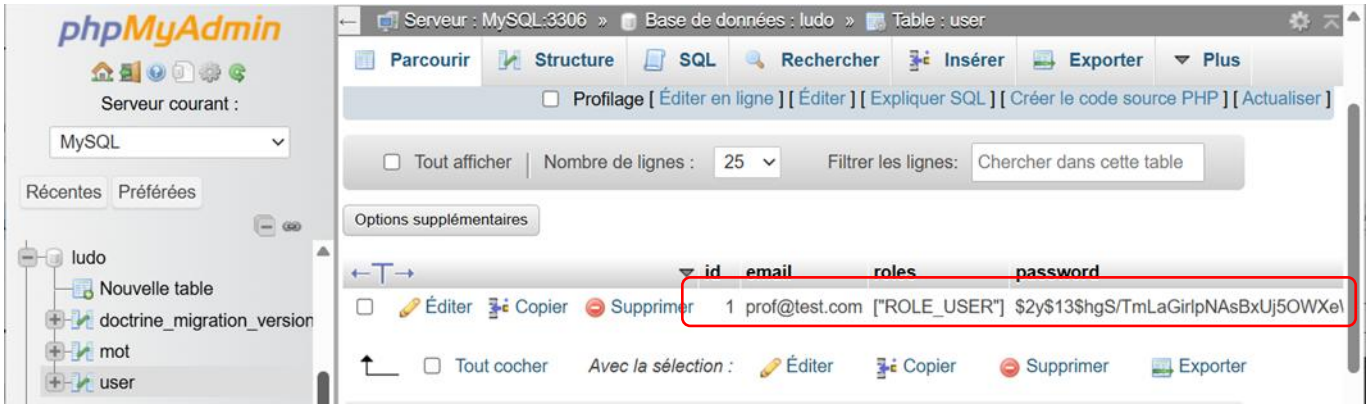
Email : prof@test.com
Role : ROLE_USER
Mot de passe : 123456
qui sera hashé

Charger les fixtures et noter la mention de Datafixtures/AuthentificationFixtures dans les fichiers pris en charge (l'option --append permet de ne pas purger la BDD et donc de conserver la table mots) :

```
php bin/console doctrine:fixtures:load --append
```

```
C:\wamp64\www\ludo-api>php bin/console doctrine:fixtures:load

 Careful, database "ludo" will be purged. Do you want to continue? (yes/no) [no]:
 > yes

   > purging database
   > loading App\DataFixtures\AppFixtures
   > loading App\DataFixtures\AuthentificationFixtures
```

Vérifier l'apparition d'un nouvel utilisateur prof@email.com avec un mot de passe hashé dans la table user :




## 3.2 Installation du bundle JWT Authentification

Le JSON Web Token (JWT) est un mécanisme d'authentification utilisé dans les API REST qui fonctionne en 3 étapes majeures :

1. Après vérification des identifiants d'un utilisateur, le serveur génère un token signé contenant des informations comme son identité et ses rôles.
2. Ce token est ensuite envoyé par le client dans l'en-tête HTTP Authorization sous la forme Bearer <token> à chaque requête.
3. Le serveur vérifie la signature et la validité du token sans stocker d'information en session (il s'agit d'une authentification *stateless*, le serveur ne conserve aucune session entre les requêtes et donc chacune doit contenir le token d'authentification).

Ce fonctionnement est particulièrement adapté aux applications web modernes et aux architectures orientées API.

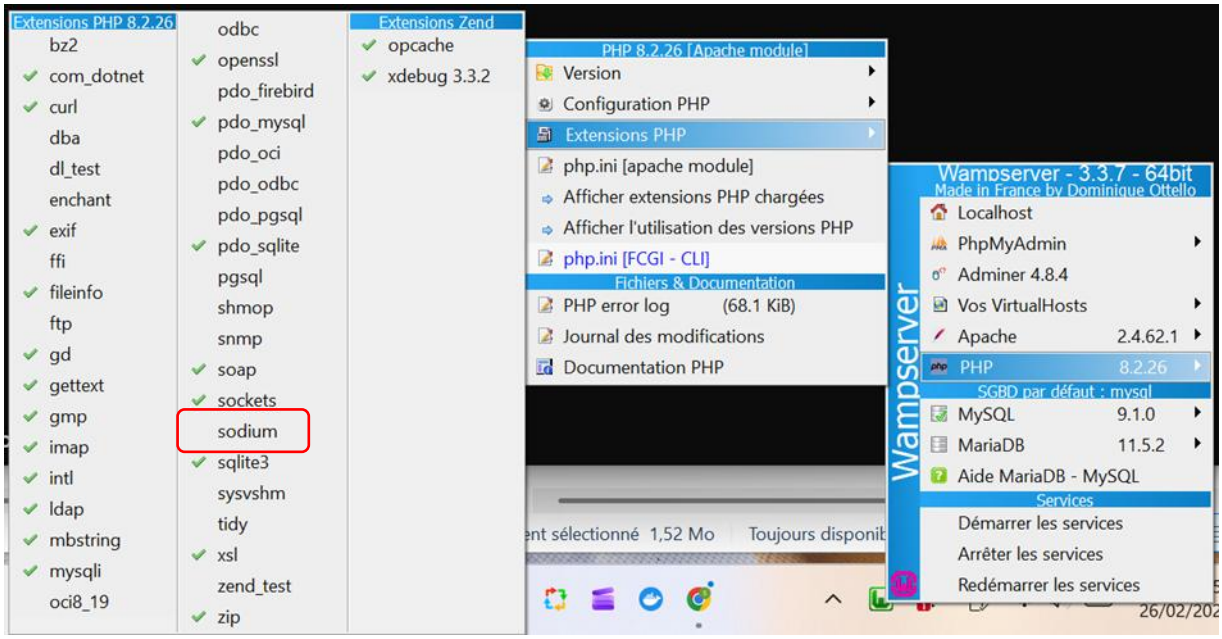
Installer le composant **JWT Authentification** qui permet de générer des clés RSA pour permettre aux consommateurs de l'API de les présenter pour s'authentifier :

Ce bundle utilise des mécanismes de sécurité forte, il nécessite :

- des signatures cryptographiques,
- des clés RSA.

PHP 8.2 inclut le module Sodium qui fournit des bibliotheque pour leur génération, mais parfois l'extension n'est pas activée dans WAMP.

Cliquer sur le logo W de Wamp dans la barre des tâches, rechercher les extensions de PHP et activer sodium :




Wamp devrait redémarrer (passage du logo de orange à vert) et afficher un message « Mise à jour des paramètres ».
Pour le vérifier, saisir la commande :

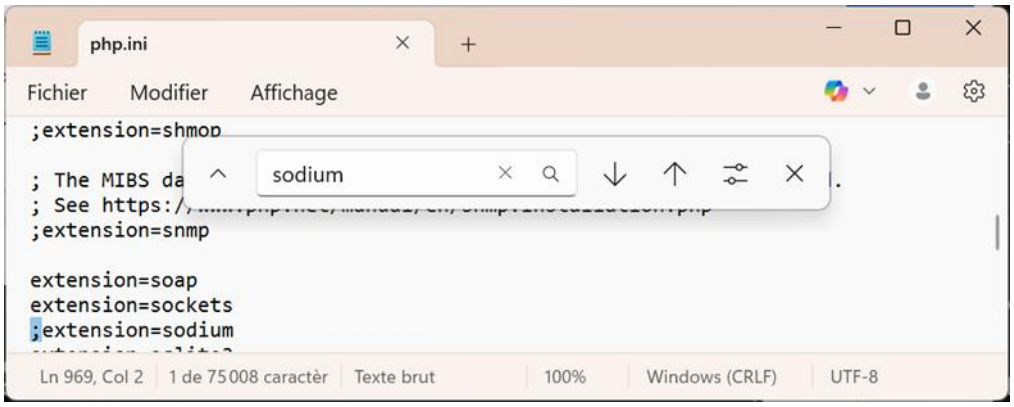
```
php -m
```

Si le module « sodium » apparait dans la liste, tout est ok, sinon, relever la version active de PHP en saisissant :

```
php -v
```

```
C:\wamp64\www\ludo-api>php -v
PHP 8.2.26 (cli) (built: Nov 19 2024 18:15:27) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.26, Copyright (c) Zend Technologies
    with Zend OPcache v8.2.26, Copyright (c), by Zend Technologies
    with Xdebug v3.3.2, Copyright (c) 2002-2024, by Derick Rethans
```

Editer le fichier c:\wamp\bin\php\phpNUMERO_DE_VERSION\php.ini et décommenter la ligne où se trouve le module sodium :




Redémarrer Wamp et constater que sodium apparait enfin dans la liste des modules de PHP (php -m) :

```
C:\Windows\System32\cmd.e
session
SimpleXML
soap
sockets
sodium
SPL
sqlite3
standard
tokenizer
xdebug
xml
xmlreader
```

Lancer l'installation du bundle JWT Authentification

```
composer require lexik/jwt-authentication-bundle
```

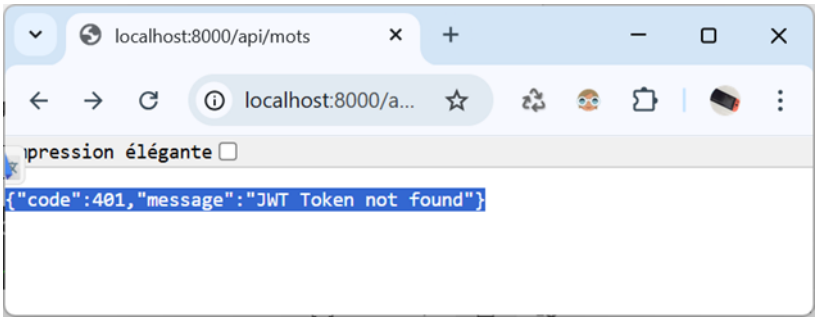
```
C:\Windows\System32\cmd.e
C:\wamp64\www\ludo-api>composer require lexik/jwt-authentication-bundle
./composer.json has been updated
Running composer update lexik/jwt-authentication-bundle
Loading composer repositories with package information
Restricting packages listed in "symfony/symfony" to "7.4.*"
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
  - Locking lcobucci/jwt (5.6.0)
  - Locking lexik/jwt-authentication-bundle (v3.2.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 2 installs, 0 updates, 0 removals
  - Downloading lcobucci/jwt (5.6.0)
  - Downloading lexik/jwt-authentication-bundle (v3.2.0)
  - Installing lcobucci/jwt (5.6.0): Extracting archive
  - Installing lexik/jwt-authentication-bundle (v3.2.0): Extracting archive
Generating autoload files
68 packages you are using are looking for funding.
Use the `composer fund` command to find out more!

Symfony operations: 1 recipe (49fd3858c9532d2173233be0032ca127)
  - Configuring lexik/jwt-authentication-bundle (>=2.5): From github.com/symfony/recipes:main
Executing script cache:clear [OK]
Executing script assets:install public [OK]

 What's next?


Some files have been created and/or updated to configure your new packages.
Please review, edit and commit them: these files are yours.
```

Adapter le fichier config/package/security.yaml

```yaml
security:
    password_hashers:
        App\Entity\User:
            algorithm: auto

    providers:
        app_user_provider:
            entity:
                class: App\Entity\User
                property: email

    firewalls:
```

```
        login:
            pattern: ^/api/login$
            stateless: true
            provider: app_user_provider
            json_login:
                check_path: /api/login
                username_path: email
                password_path: password
                success_handler: lexik_jwt_authentication.handler.authentication_success
                failure_handler: lexik_jwt_authentication.handler.authentication_failure

        api:
            pattern: ^/api
            stateless: true
            provider: app_user_provider
            jwt: ~

        dev:
            pattern: ^/(_profiler|_wdt|assets|build)/
            security: false

    access_control:
        - { path: ^/api/login$, roles: PUBLIC_ACCESS }
        - { path: ^/api/docs, roles: PUBLIC_ACCESS }
        - { path: ^/api$, roles: PUBLIC_ACCESS }
        - { path: ^/api, roles: ROLE_USER }
```

Tenter d'accéder à /api/mots :




Cela confirme que :

- la règle access_control du firewall fonctionne,
- le firewall requiert désormais un token d'authentification,
- et donc le bundle **LexikJWTAuthenticationBundle fonctionne.**

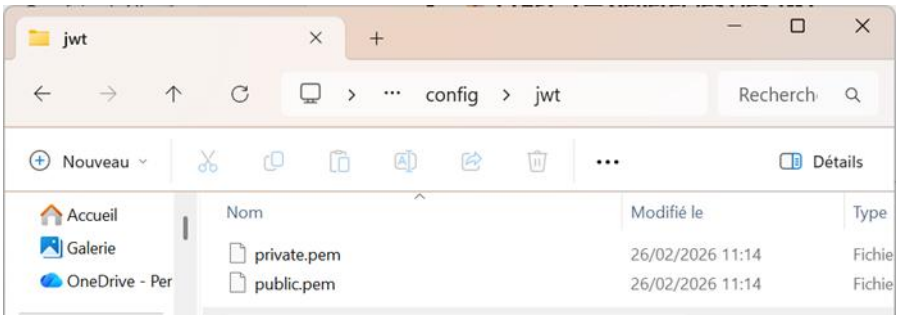
## 3.3 Génération du token d'authentification

Maintenant que JWT est installé, il est possible de générer la clé d'authentification pour les clients de l'API :

```
php bin/console lexik:jwt:generate-keypair
```

```
C:\wamp64\www\ludo-api>php bin/console lexik:jwt:generate-keypair

 [OK] Done!
```

Vérifier la création des 2 fichiers suivants dans le nouveau dossier config/jwt/ :



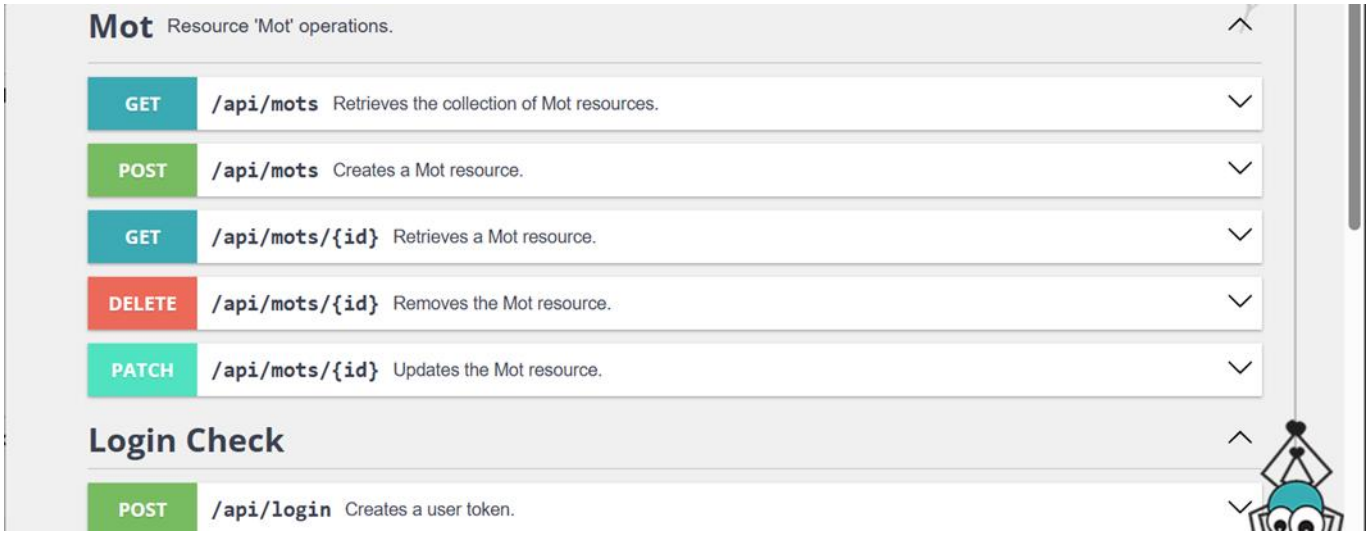

Editer le fichier config/packages/api_platform.yaml pour que API Platform utilise JWT:

```yaml
api_platform:
    title: API REST LUDO
    version: 1.0.0
    defaults:
        stateless: true
        cache_headers:
            vary: ["Content-Type", "Authorization", "Origin"]

    swagger:
        api_keys:
            JWT:
                name: Authorization
                type: header
```

vider le cache de Symfony :

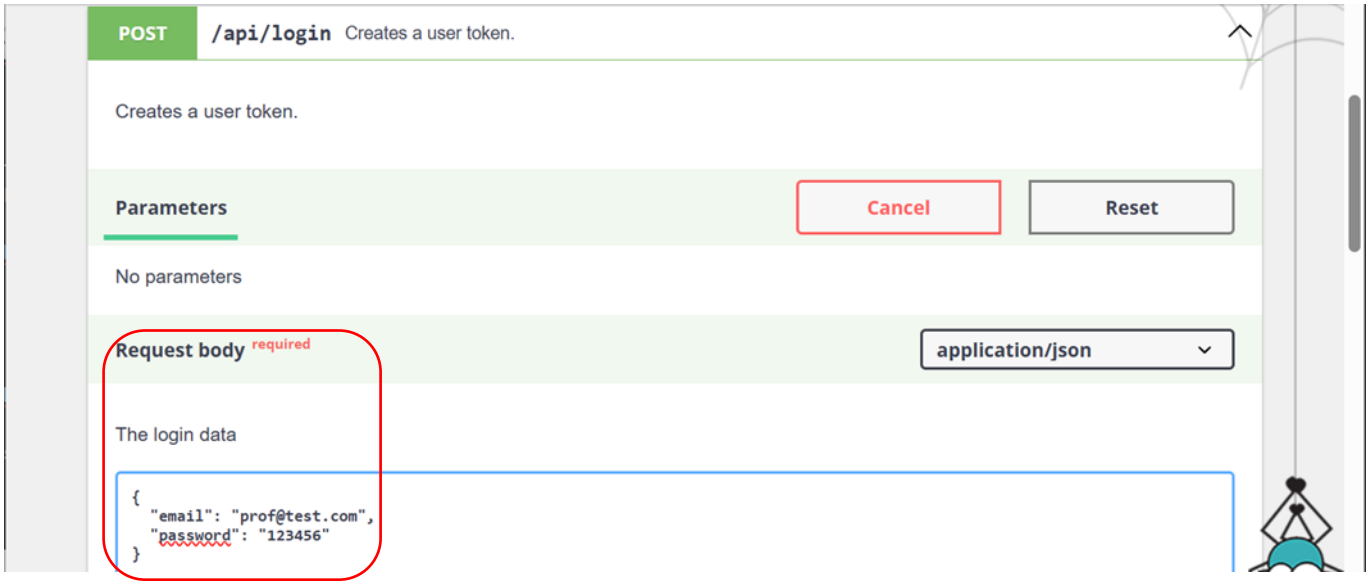
```
C:\wamp64\www\ludo-api>php bin/console cache:clear

 // Clearing the cache for the dev environment with debug true

 [OK] Cache for the "dev" environment (debug=true) was successfully cleared.
```

Aller sur API Platform, une section LOGIN est apparu :

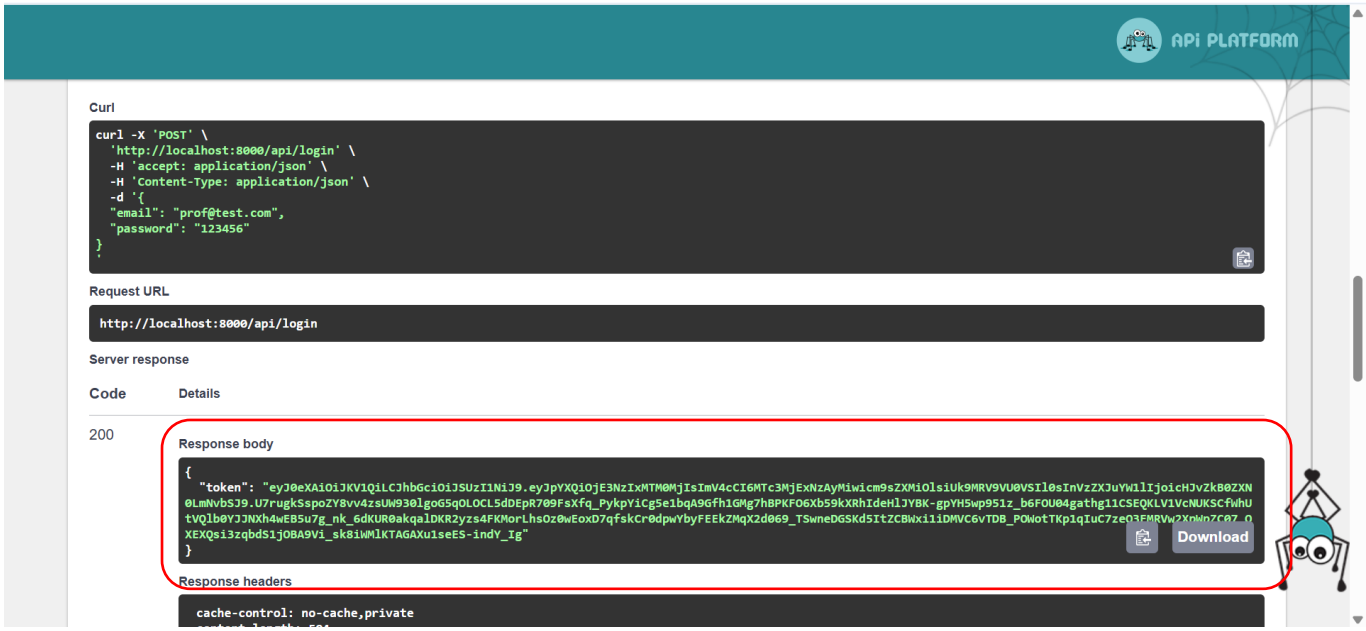



Dans « POST », cliquer sur le bouton « Try it out » et renseigner le JSON dans la section « Request body » avec le compte et le mot de passe inséré précédemment dans la table user à l'aide des fixtures :

```
{
  "email": "prof@test.com",
  "password": "123456"
}
```




Cliquer sur « Execute » et récupérer le token :




Récupérer le token au moyen d'un copier-coller, dans ce cas :

**eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJpYXQiOjE3NzIxMTM0MjIsImV4cCI6MTc3MjExNzAyMiwicm9sZXMiOlsiUk9MRV9VU0VSIl0sInVzZXJuYW1lIjoicHJvZkB0ZXN0LmNvbSJ9.U7rugkSspoZY8vv4zsUW930lgoG5qOLOCL5dDEpR709FsXfq_PykpYiCg5e1bqA9Gfh1GMg7hBPKFO6Xb59kXRhIdeHlJYBK-gpYH5wp951z_b6FOU04gathg11CSEQKLV1VcNUKSCfWhUtVQlb0YJJNXh4wEB5u7g_nk_6dKUR0akqalDKR2yzs4FKMorLhsOz0wEoxD7qfskCr0dpwYbyFEEkZMqX2d069_TSwneDGSKd5ItZCBWxi1iDMVC6vTDB_POWotTKp1qIuC7zeO3FMRVw2XpWpZC07_QXEXQsi3zqbdS1jOBA9Vi_sk8iWMlKTAGAXu1seES-indY_Ig**

copier le token

Le **JWT** (JSON Web Token) est un jeton d'authentification encodé en JSON, utilisé pour sécuriser les échanges entre un client (navigateur, application mobile…) et un serveur, notamment dans les API REST.
Il est composé de trois parties séparées par des points :

1. **le Header, qui contient :**

- le type de token (JWT),
- l'algorithme de signature utilisé (ex : RS256).

2. **le Payload, qui contient les informations :**

- l'identifiant ou l'email de l'utilisateur,
- ses rôles,
- la date d'émission (iat),
- la date d'expiration (exp).

3. **la Signature**

Permet de vérifier l'intégrité du token.
Elle garantit que le token n'a pas été modifié et qu'il provient bien du serveur.

Dans notre car, le token a une durée de vie de 1 heure
`"iat": 1772113422,`
`"exp": 1772117022`
le token a une durée de vie de 1 heure car `1772117022 - 1772113422 = 3600 secondes`

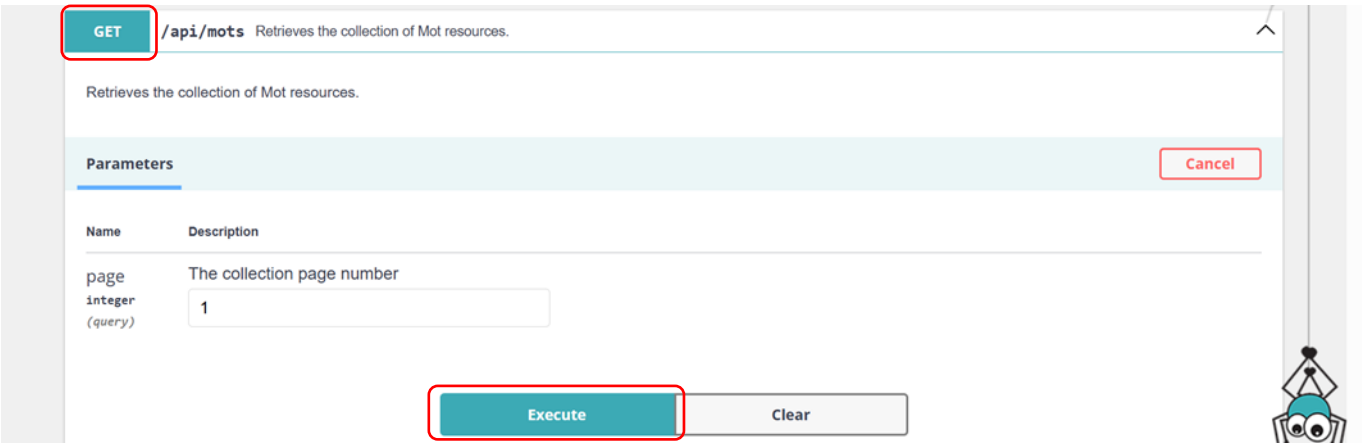
Dans cette architecture JWT, l'état de connexion n'est pas stocké sur le serveur, il est contenu dans le **token JWT lui-même**, qui est :

- stocké côté client (navigateur, application mobile…),
- envoyé dans chaque requête via l'en-tête Authorization.

Le serveur est en mode stateless, il se contente de vérifier la validité du token à chaque appel et ne conserve aucune session entre les requêtes.

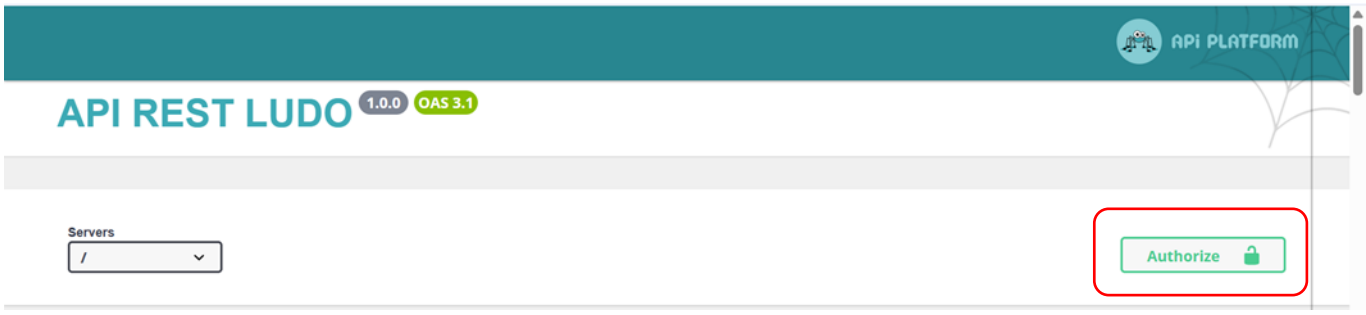
## 3.4 Tester une consommation de l'API en passant le token

Faire un nouveau test de GET



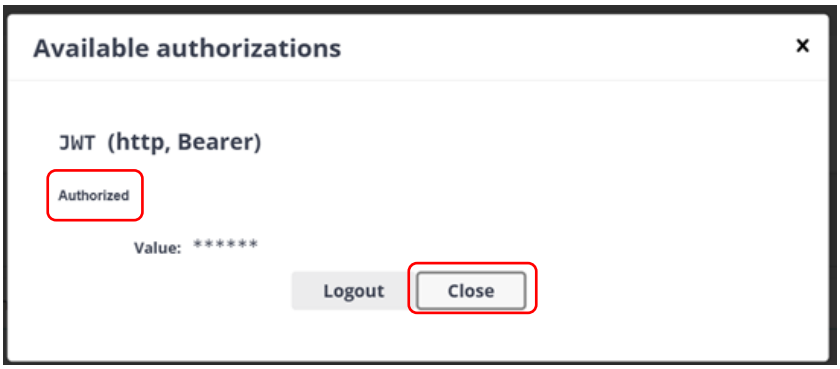

Cela produit une réponse 401 « Error Unauthorized » puisque le token n'a pas été soumis en même temps que la requête GET :

Cliquer sur le bouton « Authorize »

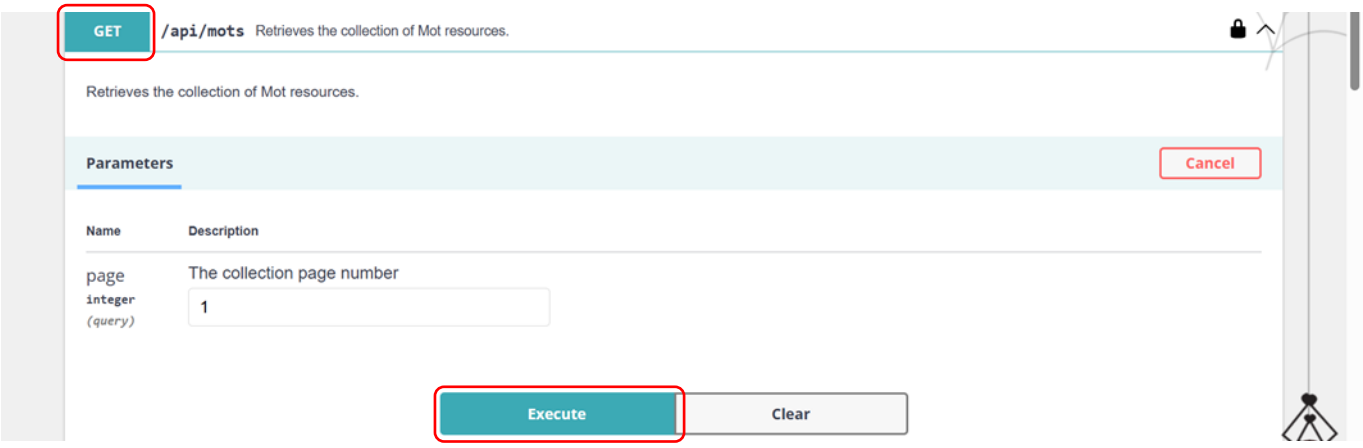



Copier-coller le token généré précédemment (attention à ne pas copier de caractère retour chariot à la fin) :

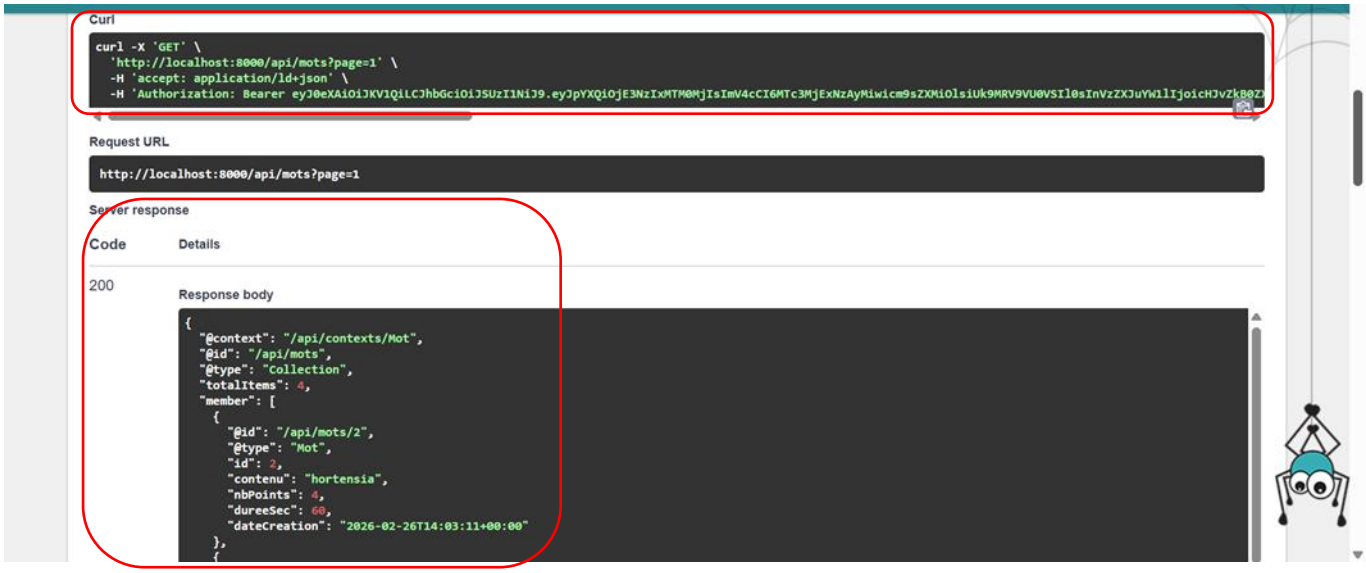
cliquer sur le bouton « Authorize » et relever le statut « Authorized » :




Cliquer sur le bouton close de la fenêtre Available authorizations, puis cliquer de nouveau sur GET + le bouton « Execute » :




Relever la commande curl qui intègre le token et la réponse 200 présentant la liste des mots :



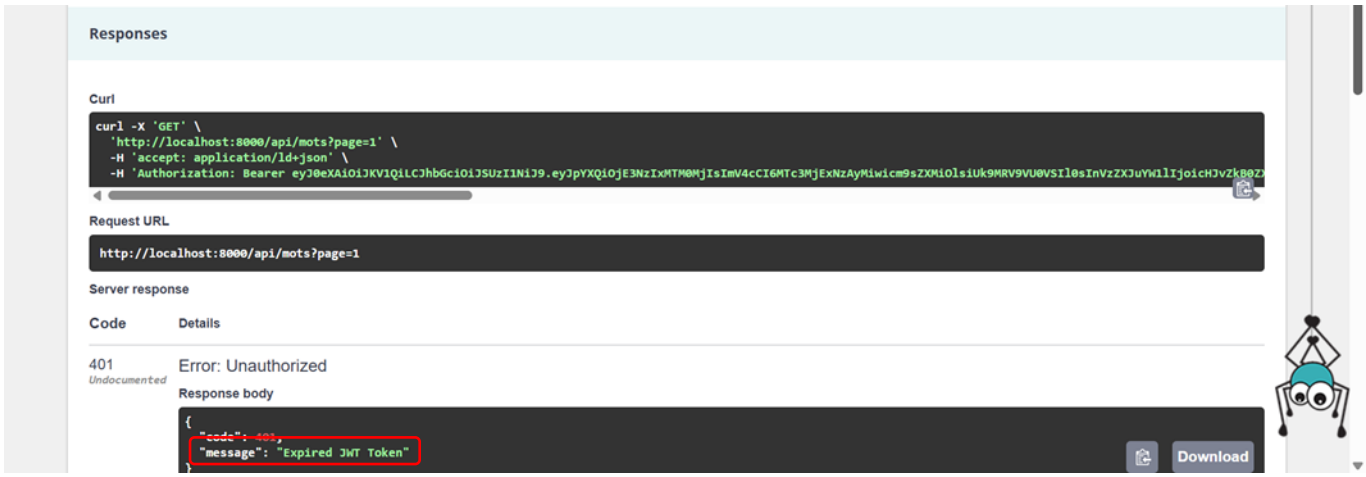

Il est aussi possible d'interroger l'API en ligne de commandes :

```
curl -X GET http://localhost:8000/api/mots -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJpYXQiOjE3NzIxMTM0MjIsImV4cCI6MTc3MjExNzAyMiwicm9sZXMiOlsiUk9MRV9VU0VSIl0sInVzZXJuYW1lIjoicHJvZkB0ZXN0LmNvbSJ9.U7rugkSspoZY8vv4zsUW930lgoG5qOLOCL5dDEpR709FsXfq_PykpYiCg5e1bqA9Gfh1GMg7hBPKFO6Xb59kXRhIdeHlJYBK-gpYH5wp951z_b6FOU04gathg11CSEQKLV1VcNUKSCfWhUtVQlb0YJJNXh4wEB5u7g_nk_6dKUR0akqalDKR2yzs4FKMorLhsOz0wEoxD7qfskCr0dpwYbyFEEkZMqX2d069_TSwneDGSKd5ItZCBWxi1iDMVC6vTDB_POWotTKp1qIuC7zeO3FMRVw2XpWpZC07_QXEXQsi3zqbdS1jOBA9Vi_sk8iWMlKTAGAXu1seES-indY_Ig"
```

Constater la liste des mots dans la réponse donnée par l'API :

```
C:\wamp64\www\ludo-api>curl -X GET http://localhost:8000/api/mots -H "Authorization: Bearer eyJ0
eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJpYXQiOjE3NzIxMTM0MjIsImV4cCI6MTc3MjExNzAyMiwicm9sZXMiOlsiUk9
MRV9VU0VSIl0sInVzZXJuYW1lIjoicHJvZkB0ZXN0LmNvbSJ9.U7rugkSspoZY8vv4zsUW930lgoG5qOLOCL5dDEpR709FsX
fq_PykpYiCg5e1bqA9Gfh1GMg7hBPKFO6Xb59kXRhIdeHlJYBK-gpYH5wp951z_b6FOU04gathg11CSEQKLV1VcNUKSCfWhU
tVQlb0YJJNXh4wEB5u7g_nk_6dKUR0akqalDKR2yzs4FKMorLhsOz0wEoxD7qfskCr0dpwYbyFEEkZMqX2d069_TSwneDGSK
d5ItZCBWxi1iDMVC6vTDB_POWotTKp1qIuC7zeO3FMRVw2XpWpZC07_QXEXQsi3zqbdS1jOBA9Vi_sk8iWMlKTAGAXu1seES
-indY_Ig"
{"@context":"\/api\/contexts\/Mot","@id":"\/api\/mots","@type":"Collection","totalItems":4,"memb
er":[{"@id":"\/api\/mots\/2","@type":"Mot","id":2,"contenu":"hortensia","nbPoints":4,"dureeSec":
60,"dateCreation":"2026-02-26T14:03:11+00:00"},{"@id":"\/api\/mots\/3","@type":"Mot","id":3,"con
tenu":"véhiculer","nbPoints":3,"dureeSec":60,"dateCreation":"2026-02-26T14:03:11+00:00"},{"@id":
"\/api\/mots\/4","@type":"Mot","id":4,"contenu":"parentalité","nbPoints":3,"dureeSec":60,"dateCr
eation":"2026-02-26T14:03:53+00:00"},{"@id":"\/api\/mots\/5","@type":"Mot","id":5,"contenu":"ras
sembler","nbPoints":2,"dureeSec":60,"dateCreation":"2026-02-26T14:03:53+00:00"}]}
C:\wamp64\www\ludo-api>
```

Relancer un test de GET 1 heure après, API Platforrm devrait naturellement indiquer que le token est expiré :



## 3.5 Gestion des droits d'accès

L'objectif à cette étape et de faire une gestion des accès pour que :

- Le GET soit accessible aux rôles ROLE_USER et ROLE_ADMIN,
- Le POST soit accessible au rôle ROLE_ADMIN uniquement,
- Le PATCH soit accessible au rôle ROLE_ADMIN uniquement,
- Le DELETE soit accessible au rôle ROLE_ADMIN uniquement.

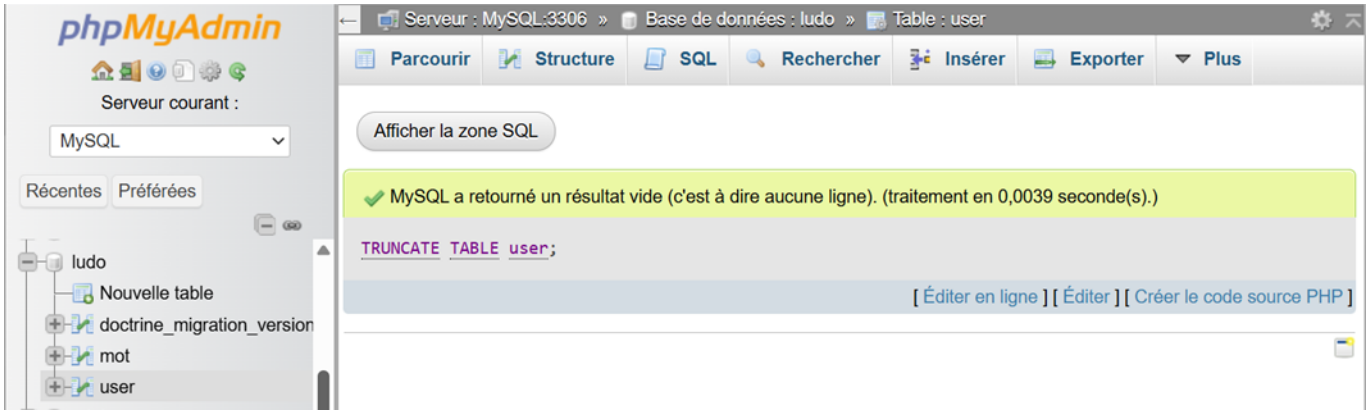
***Création d'un compte administrateur***

Créer un nouvel utilisateur admin@mail.com avec le rôle ROLE_ADMIN en utilisant les fixtures.
Modifier le fichier src/DataFixtures/AuthentificationFixtures.php en ajoutant les lignes suivantes pour créer l'admin :

```
        $manager->persist($user);

        $admin = new User();
        $admin->setEmail('admin@mail.com');
        $admin->setRoles(['ROLE_ADMIN', 'ROLE_USER']);

        $hashedPasswordAdmin = $this->passwordHasher->hashPassword(
            $admin,
            'admin123'
        );

        $admin->setPassword($hashedPasswordAdmin);
        $manager->persist($admin);

        $manager->flush();
```

Vider le contenu de la table user, pour cela, passer cette commande en BDD :

```
TRUNCATE TABLE user;
```

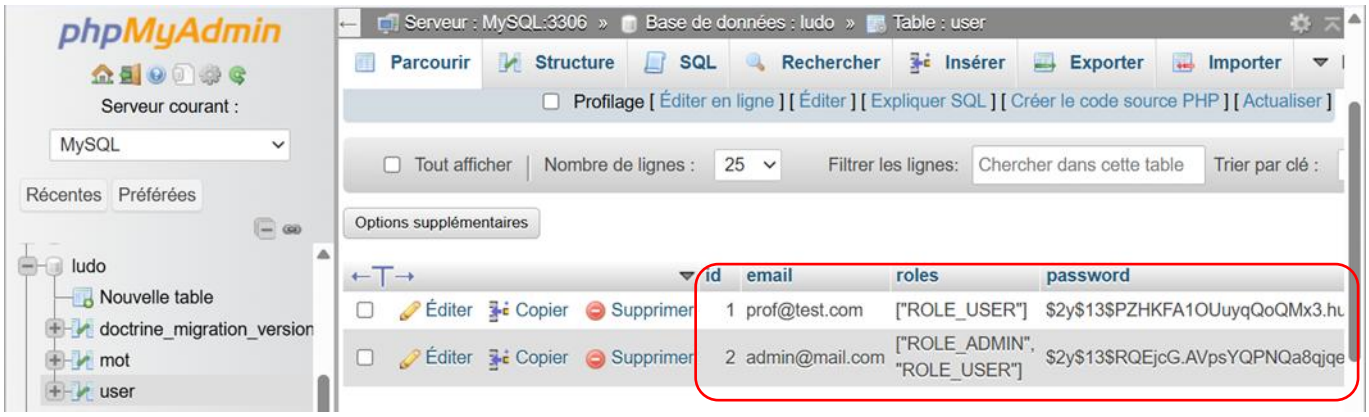



Puis recharger les fixtures (l'option --append, ne purge pas la BDD, la table mot sera conservée) :

```
php bin/console doctrine:fixtures:load –append
```

```
C:\wamp64\www\ludo-api>php bin/console doctrine:fixtures:load --append
   > loading App\DataFixtures\AppFixtures
   > loading App\DataFixtures\AuthentificationFixtures
```

Vérifier le rechargement des 2 comptes avec les profils ROLE_USER et ROLE_ADMIN dans la table users :




***Prise en compte du nouveau rôle ROLE_ADMIN pour l'acces aux end points POST, PATCH et DELETE***

Comme la section access_control du fichier security.yaml contient :

```
    access_control:
        - { path: ^/api/login$, roles: PUBLIC_ACCESS }
        - { path: ^/api/docs, roles: PUBLIC_ACCESS }
        - { path: ^/api$, roles: PUBLIC_ACCESS }
        - { path: ^/api, roles: ROLE_USER }
```

il n'y a rien à ajouter car Symfony reconnaît automatiquement tous les rôles commençant par « ROLE_... »

Modifier le fichier src/Entity/Mot.php en chargeant les bibliothèques suivantes :

```
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;
use ApiPlatform\Metadata\Post;
use ApiPlatform\Metadata\Patch;
use ApiPlatform\Metadata\Delete;
use App\Repository\MotRepository;
```

et remplacer :

```
#[ApiResource]
```

par :

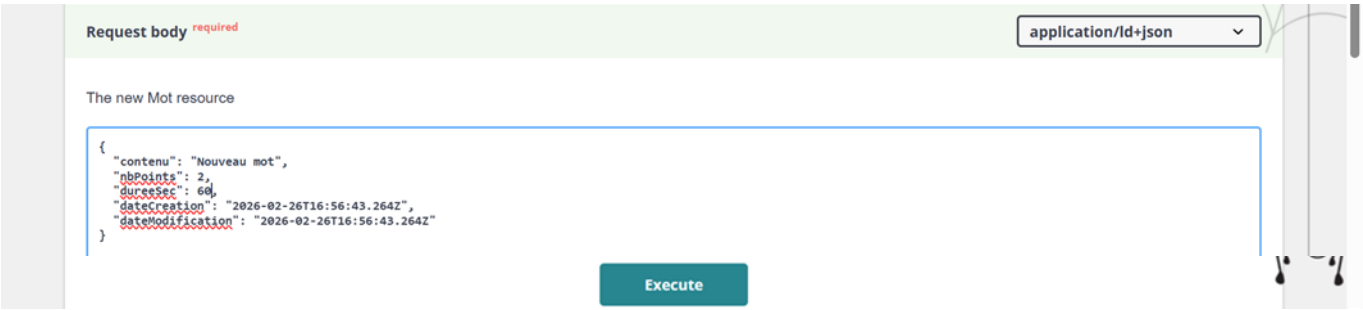
```
#[ApiResource(
    operations: [
        new GetCollection(
            security: "is_granted('ROLE_USER')"
        ),
        new Get(
            security: "is_granted('ROLE_USER')"
        ),
        new Post(
            security: "is_granted('ROLE_ADMIN')"
        ),
        new Patch(
            security: "is_granted('ROLE_ADMIN')"
        ),
        new Delete(
```

```
            security: "is_granted('ROLE_ADMIN')"
        )
    ]
)]
```

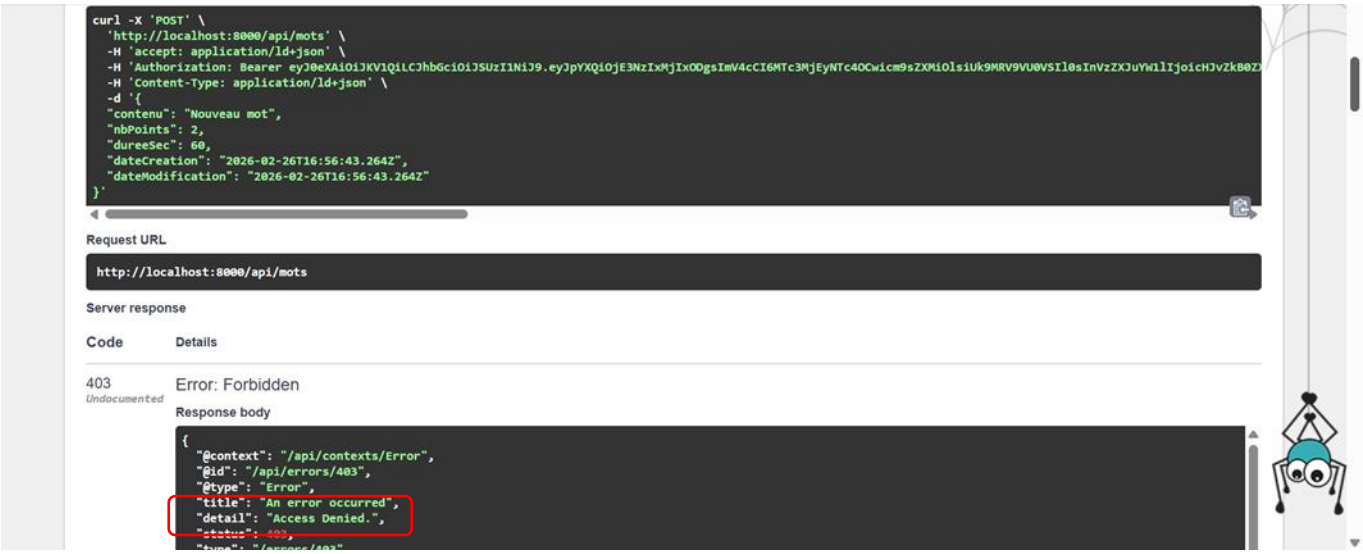
Vide le cache de Symfony

```
C:\wamp64\www\ludo-api>php bin/console cache:clear

 // Clearing the cache for the dev environment with debug true

 [OK] Cache for the "dev" environment (debug=true) was successfully cleared.
```

Tester un POST avec le profil ROLE_USER




Le POST est refusé, le message 403 précise « acces denied ».

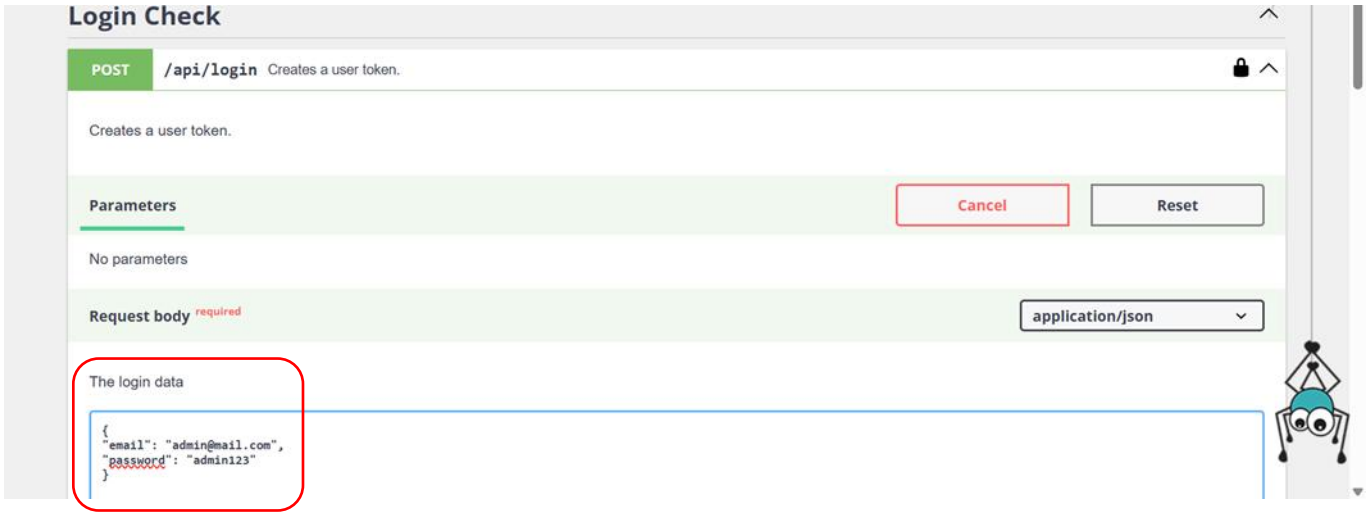



Vérifier que le GET est toujours accessible au role ROLE_USER :;

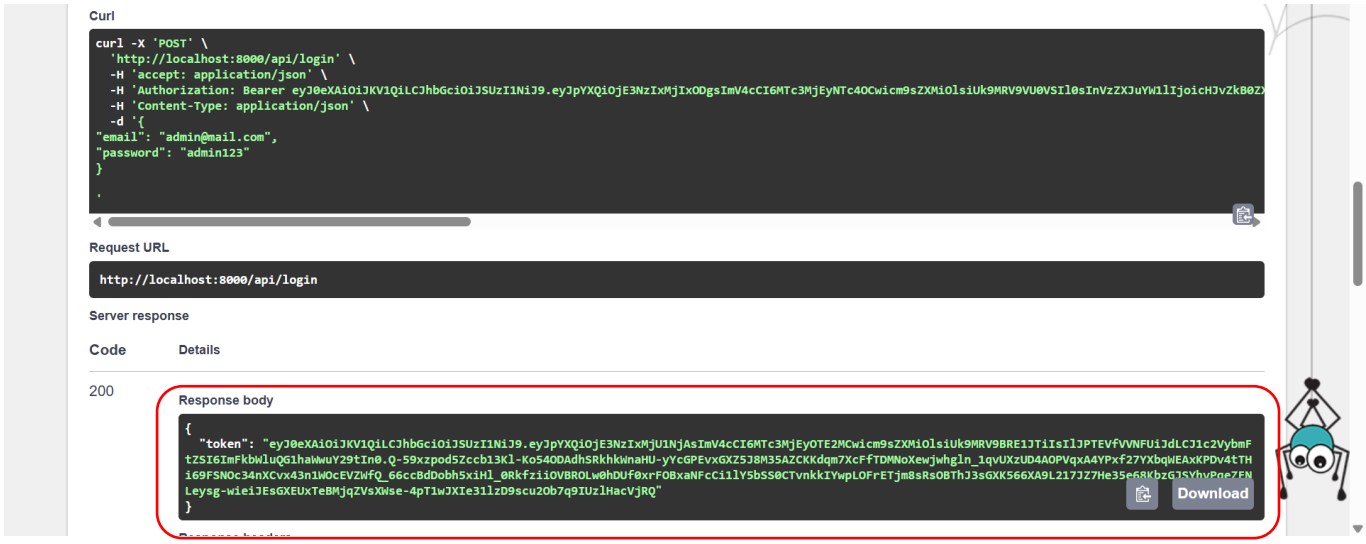
Générer un token pour l'utilisateur administrateur.

Cliquer sur le ENDPOINT LOGIN et saisir le JSON suivant dans le champ :

```
{
"email": "admin@mail.com",
"password": "admin123"
}
```




Récupérer le token généré pour l'utilisateur admin@mail.com

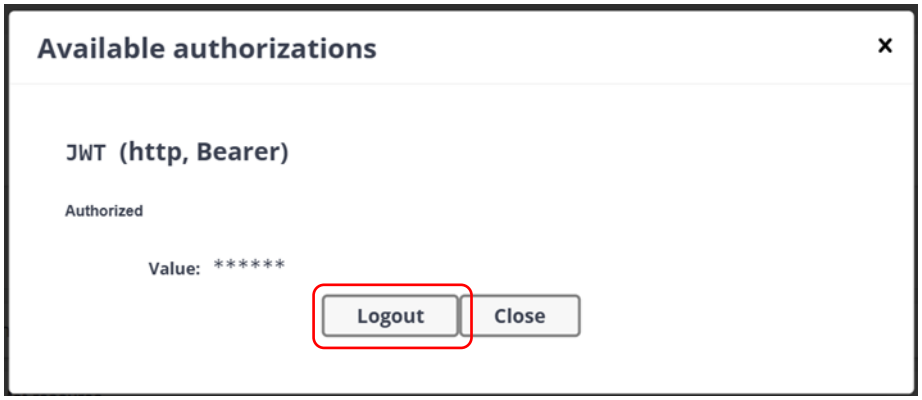


Dans ce cas :

**eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJpYXQiOjE3NzIxMjU1NjAsImV4cCI6MTc3MjEyOTE2MCwicm9sZXMiOlsiUk9MRV9BRE1JTiIsIlJPTEVfVVNFUiJdLCJ1c2VybmFtZSI6ImFkbWluQG1haWwuY29tIn0.Q-59xzpod5Zccb13Kl-Ko54ODAdhSRkhkWnaHU-yYcGPEvxGXZ5J8M35AZCKKdqm7XcFfTDMNoXewjwhgln_1qvUXzUD4AOPVqxA4YPxf27YXbqWEAxKPDv4tTHi69FSNOc34nXCvx43n1WOcEVZWfQ_66ccBdDobh5xiHl_0RkfziiOVBROLw0hDUf0xrFOBxaNFcCi1lY5bSS0CTvnkkIYwpLOFrETjm8sRsOBThJ3sGXK566XA9L217JZ7He35e68KbzGJSYhvPqeZFNLeysg-wieiJEsGXEUxTeBMjqZVsXWse-4pT1wJXIe31lzD9scu2Ob7q9IUzlHacVjRQ**

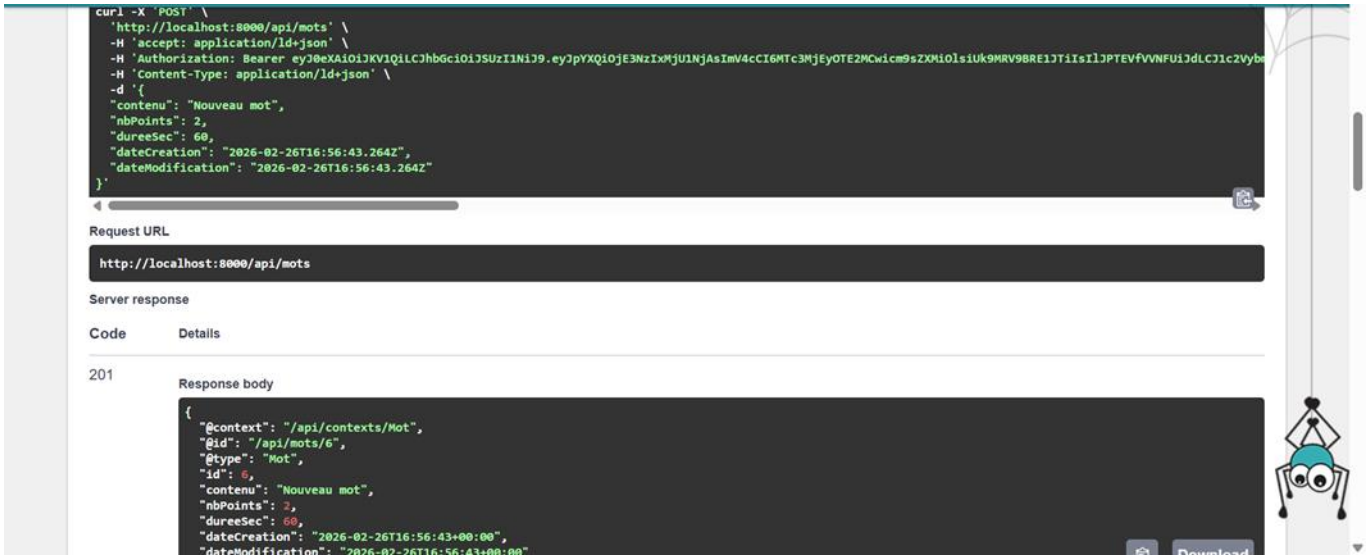
Cliquer sur le bouton « Authorize » :

Cliquer sur le bouton « Logout »

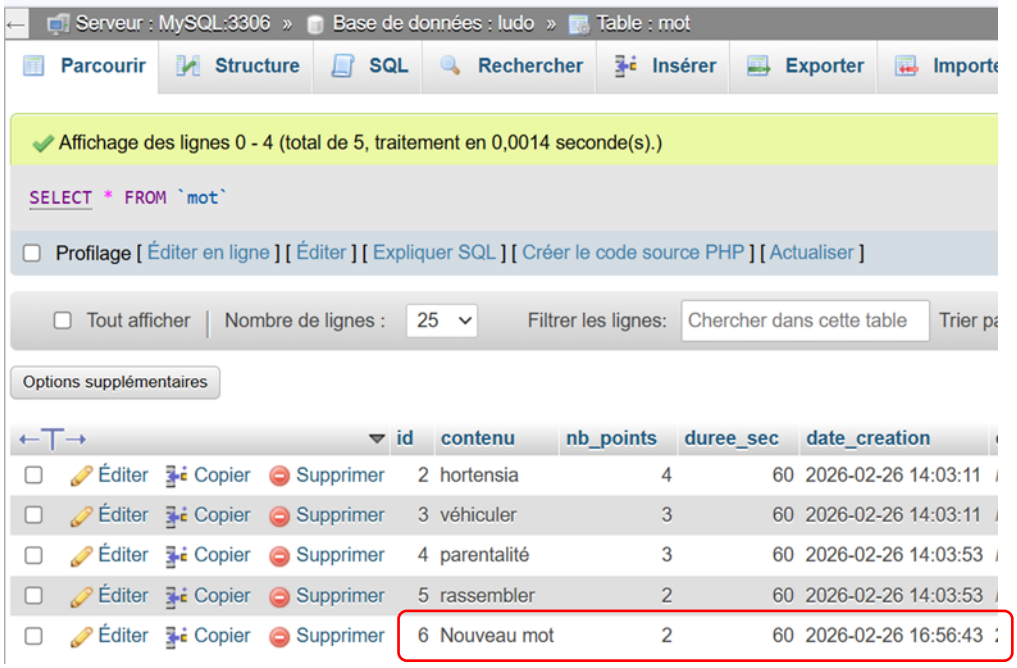



et saisir ce token :

Puis tester un POST avec le profil ROLE_ADMIN :

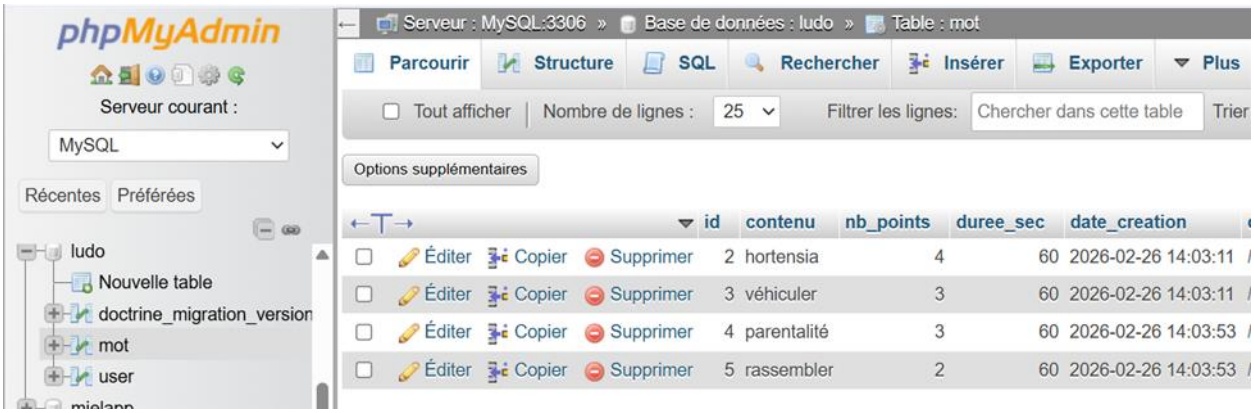



Vérifier la création en BDD :




Tester le DELETE avec le compte de l'administrateur :

Vérifier la suppression en BDD :




Vérifier que l'administrateur a également accès au GET :

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/mots?page=1' \
  -H 'accept: application/ld+json' \
  -H 'Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJpYXQiOjE3NzIxMjU1NjAsImV4cCI6MTc3MjEyOTE2MCwicm9sZXMiOlsiUk9MRV9BRE1JTiIsIlJPTEVfVVNFUiJdLCJ1c2Vyb
```

Request URL

```
http://localhost:8000/api/mots?page=1
```

Server response

| Code | Details |
|---|---|
| 200 | Response body |


```
{
  "@context": "/api/contexts/Mot",
  "@id": "/api/mots",
  "@type": "Collection",
  "totalItems": 4,
  "member": [
    {
      "@id": "/api/mots/2",
      "@type": "Mot",
      "id": 2,
      "contenu": "hortensia",
      "nbPoints": 4,
      "dureeSec": 60,
      "dateCreation": "2026-02-26T14:03:11+00:00"
    },
```

## 4. Conclusion

Cette activité a permis de mettre en place la sécurisation complète d'une API REST développée avec Symfony et API Platform.
Une authentification stateless basée sur le protocole JWT a été implémentée, garantissant que seules les requêtes munies d'un token valide puissent accéder aux ressources.
Le composant Security a été configuré afin de gérer les rôles utilisateurs et de restreindre certaines opérations aux administrateurs.

Enfin, cette activité illustre les bonnes pratiques de sécurisation des services web modernes et l'importance du contrôle d'accès dans une architecture orientée API.